

面向 LLM / Agent Systems 科研的深度学习零基础课

第 02 讲

实验设计：Tensor 索引、切片、Reshape 与 Transpose

实验目标：通过可重复的小实验验证每种 Tensor 操作怎样改变 Shape，并建立“先预测、后运行、再解释”的科研式调试习惯。

实验设计文档

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

1 实验名称

Experiment 02: 3D Tensor 的索引、切片、重塑与轴交换实验

2 实验背景

在深度学习代码中，很多错误并不是模型理论错误，而是数据轴的处理错误。例如：

- 本来想取一个 Token，却把 batch 轴删掉；
- 本来想交换 sequence 和 hidden，结果误用了 reshape；
- batch size = 1 时，无参数 squeeze 把 batch 维度删除；
- Multi-Head Attention 中轴顺序不符合后续矩阵乘法要求。

因此，本实验不追求复杂网络训练，而是把一个小型 3D Tensor 当成“实验对象”，逐步改变其结构并观察结果。

科研视角

本实验的真正目的不是记住 NumPy API，而是训练一个以后科研调试非常重要的动作：**在运行前先写出预期 Shape，然后用程序验证。如果预测和实测不一致，必须解释原因。**

3 研究问题

1. 使用整数索引和使用切片时，为什么输出 ndim 可能不同？
2. reshape 在什么条件下合法？它是否改变元素总数？
3. transpose 与 reshape 即使得到相同 Shape，数值布局是否相同？
4. 对 [B,T,D] Tensor，怎样准确取出“所有样本的最后一个 Token”？
5. 增加或删除长度为 1 的轴时，数据值是否发生改变？

4 实验对象

主实验 Tensor：

```
X = np.arange(24).reshape(2, 3, 4)
```

其 Shape：

(2, 3, 4)

本实验约定轴语义：

$[B, T, D] = [batch, sequence, hidden]$

也就是：

- 2 个样本；
- 每个样本 3 个 Token；
- 每个 Token 4 个 hidden features。

5 实验假设

运行代码之前，先判断下面假设是否合理：

1. H1: `X[0]` 会删除 batch 轴，输出 Shape 为 `(3,4)`。
2. H2: `X[0:1]` 会保留 batch 轴，输出 Shape 为 `(1,3,4)`。
3. H3: `X[:,1,:]` 的输出 Shape 为 `(2,4)`，而 `X[:,1:2,:]` 为 `(2,1,4)`。
4. H4: `X.reshape(6,4)` 合法，因为元素总数仍为 24。
5. H5: `X.transpose(0,2,1)` 的 Shape 为 `(2,4,3)`。
6. H6: 对同一个 2D 数组，从 `(2,3)` 变成 `(3,2)` 时，`reshape` 和 `transpose` 得到的数值布局不同。
7. H7: 给 `(3,4)` 增加 batch 轴后得到 `(1,3,4)`，元素总数仍然不变。

6 变量设计

类别	本实验定义
实验对象	固定的 NumPy Tensor <code>X=np.arange(24).reshape(2,3,4)</code>
自变量	Tensor 操作： <code>index</code> 、 <code>slice</code> 、 <code>reshape</code> 、 <code>transpose</code> 、 <code>expand_dims</code> 、 <code>squeeze</code>
主要观测变量	输出 <code>value</code> 、 <code>ndim</code> 、 <code>shape</code> 、 <code>size</code>
控制变量	原始数据元素 0–23、NumPy 环境、操作顺序
核心判断	预测 Shape 是否与实测一致；操作是否改变元素数量；轴语义是否发生变化

7 实验环境

最低要求：

- Python 3.x；
- NumPy；
- VS Code、Jupyter 或终端任意一种运行方式。

检查环境：

```
python -c "import numpy as np; print(np.__version__)"
```

8 实验一：整数索引是否会减少维度

8.1 操作

```
a = X[0]
b = X[0:1]
```

8.2 运行前预测

表达式	预测 Shape	解释
X[0]		
X[0:1]		

8.3 要验证的现象

两种操作都只保留第 0 个样本，但结构不同：

- 整数索引固定 axis 0 的一个位置，因此 axis 0 消失；
- 切片仍然保留 axis 0，只是长度变成 1。

9 实验二：只取一个 Token 时，是否保留 sequence 轴

```
a = X[:, 1, :]  
b = X[:, 1:2, :]
```

预测：

$a.shape = (2, 4)$

$b.shape = (2, 1, 4)$

然后比较：

- 两个结果包含的数值是否对应同一个 Token；
- ndim 是否不同；
- 哪个结果还显式保留了 sequence 轴。

10 实验三：模拟 LLM 中“最后一个 Token”

执行：

```
last_token = X[:, -1, :]
```

预测 Shape：

(2, 4)

请解释：

- 为什么 batch 轴仍然存在；
- 为什么 sequence 轴消失；
- 为什么 hidden 轴长度仍然是 4。

与 LLM / Transformer / Agent Systems 的连接

如果真实 LLM logits 为 (B,T,V)，同样的 logits[:, -1, :] 会得到 (B,V)，即每个样本最后一个位置对整个词表的 logits。

11 实验四：Reshape 的元素总数守恒

执行：

```
a = X.reshape(6, 4)
b = X.reshape(4, 6)
c = X.reshape(2, 3, -1)
```

记录：

对象	Shape	size	是否成功
原 X	(2, 3, 4)	24	是
reshape(6, 4)			
reshape(4, 6)			
reshape(2, 3, -1)			

然后尝试一个非法操作：

```
X.reshape(2, 4, 4)
```

预期：报错，因为目标需要 32 个元素，而原 Tensor 只有 24 个。

常见误区 / 风险

这个报错是有价值的。它说明框架在保护“元素总数守恒”。以后遇到 reshape 错误时，第一件事是计算新旧 Shape 的元素总数。

12 实验五：Reshape 与 Transpose 的对照实验

构造：

```
A = np.array([[1, 2, 3],
               [4, 5, 6]])
```

分别计算：

```
A_reshape = A.reshape(3, 2)
A_transpose = A.T
```

两者 Shape 都是：

(3, 2)

但预期数值：

$$A_{\text{reshape}} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

科研视角

这一步是本讲最重要的实验之一。它证明：**只比较 Shape 不足以判断两个 Tensor 操作是否等价**。在论文复现和模型改写中，还必须检查轴语义和数值对应关系。

13 实验六：增加和删除 batch 轴

取第 0 个样本：

```
sample = X[0]
```

其 Shape：

(3, 4)

增加 batch 轴：

```
sample_with_batch = np.expand_dims(sample, axis=0)
```

预期：

$(1, 3, 4)$

再删除这个长度为 1 的轴：

```
restored = np.squeeze(sample_with_batch, axis=0)
```

预期：

$(3, 4)$

最后比较：

```
np.array_equal(sample, restored)
```

预期结果为 True。

14 实验七：边界风险 - 无参数 squeeze

构造：

```
y = np.zeros((1, 3, 1, 4))
```

执行：

```
np.squeeze(y)
```

预测：

$(1, 3, 1, 4) \rightarrow (3, 4)$

因为两个长度为 1 的轴都会被删除。

请思考：如果第一个 1 是 batch 轴，第三个 1 是某个必须保留的 channel / head 轴，这样做是否总是安全？

15 实验记录表

实验完成后填写：

操作	预测 Shape	实测 Shape	是否理解原因
X[0]			
X[0:1]			
X[:,1,:]			
X[:,1:2,:]			
X[:, -1,:]			
X.reshape(6,4)			
X.transpose(0,2,1)			

16 实验结果应该支持的结论

正常情况下，本实验应支持以下结论：

- 1. Integer indexing 和 slicing 即使取到相同的数值范围，也可能产生不同 ndim；
- 2. reshape 不改变 size，但会改变元素的分组方式；
- 3. transpose 不改变 size 和 ndim，但会改变轴的排列顺序；
- 4. reshape 与 transpose 不能因为最终 Shape 相同就视为等价；
- 5. expand_dims / squeeze 主要改变结构，不增加或删除真实数值；
- 6. [B,T,D] 这种轴语义可以直接指导 LLM Tensor 的索引方式。

17 失败排查

17.1 1. reshape 报 ValueError

先计算：

$$\text{old size} = \prod \text{old shape}$$

和：

$$\text{new size} = \prod \text{new shape}$$

是否相等。

17.2 2. 输出 Shape 比预期少一维

检查是否使用了整数索引，例如 X[0]、X[:,1,:]。如果想保留轴，应考虑切片：0:1、1:2。

17.3 3. transpose 后数值顺序 “看起来乱了”

这通常不是错误。transpose 的目的就是改变轴顺序，所以二维打印布局当然会发生改变。关键是确认每个轴的新语义。

17.4 4. squeeze 报错

如果指定的 axis 长度不等于 1，NumPy 不允许 squeeze。先打印 `x.shape`。

18 实验后的科研式问题

1. 如果一个模型复现代码中，论文要求 `[B,T,D]`，而代码实际输出 `[B,D,T]`，为什么即使 size 完全相同也不能认为没问题？
2. 为什么“代码没有报错”不是 Tensor 结构正确的充分条件？
3. 如果某个 Ablation 替换模块后输出 Shape 改变，为什么可能导致实验比较不公平？
4. 在 Multi-Head Attention 中，把 `[B,T,H,Dh]` 变成 `[B,H,T,Dh]` 的本质是什么？

19 验收标准

本节检查点

本实验通过标准不是“脚本运行成功”，而是你能够在不运行代码的情况下，正确预测至少 80% 的 Shape，并解释：

1. index 与 slice 对维度的区别；
2. reshape 与 transpose 的区别；
3. expand_dims / squeeze 为什么不改变真实元素数；
4. `X[:, -1, :]` 在 `[B,T,D]` 中表示什么。